

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

*I **KESHAVARAM L/192511253** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

L. Keshavam

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SOLUTIONS:

Q1. School Management System-Students with Class Teacher and Grade

Scenario: A school needs a consolidated view of each student, their class's teacher, and their current grade.

Understanding of Scenario: Three concepts are entangled in one sentence: a student belongs to a class, a class has one class-teacher, and a student has a grade. This means at least three relations are needed and none of the requested facts live in a single table — they must be joined.

Key Issues Identified: 1) A class and its teacher must be modelled separately from Student, since many students share one class teacher. 2) "Grade" is a per-student attribute of that term (e.g. overall grade obtained), not a property of the class. 3) Joining three tables correctly requires two separate FK relationships (Student→Class and Class→Teacher).

```
TEACHER(TeacherID PK, TeacherName)
CLASS(ClassID PK, ClassName, ClassTeacherID FK->TEACHER)
STUDENT(SID PK, SName, ClassID FK->CLASS, Grade)
```

Application of Concepts — Schema:

Sample data — Teacher

TeacherID	TeacherName
T1	Mrs. Kalpana
T2	Mr. Ramesh

Sample data — Class

ClassID	ClassName	ClassTeacherID
C1	Grade 10-A	T1
C2	Grade 10-B	T2

Sample data — Student

SID	SName	ClassID	Grade
S1	Aravind	C1	A
S2	Bhavana	C1	B
S3	Chetan	C2	A

```
SELECT S.SName, C.ClassName, T.TeacherName AS ClassTeacher, S.Grade
FROM Student S
JOIN Class C ON S.ClassID = C.ClassID
JOIN Teacher T ON C.ClassTeacherID = T.TeacherID;
```

Solution:

Result

SName	ClassName	ClassTeacher	Grade
Aravind	Grade 10-A	Mrs. Kalpana	A

Bhavana	Grade 10-A	Mrs. Kalpana	B
Chetan	Grade 10-B	Mr. Ramesh	A

Decision Making: INNER JOIN was chosen (not OUTER) because every student is expected to belong to exactly one class and every class to have exactly one teacher in this scenario — there is no legitimate case of an orphan student, so INNER JOIN's stricter matching is the correct, simpler choice rather than defensively using LEFT JOIN.

Clarity: The final result reads as one flat, human-readable row per student — name, class, teacher, grade — which is exactly the consolidated view the school office would need to print or export.

Q2. Hospital Appointments -Doctors With >10 Appointments This Month

Scenario: A hospital wants to identify doctors who are heavily booked in the current month, to help with workload balancing.

Understanding of Scenario: This needs two things simultaneously: aggregation (counting appointments per doctor) and a time filter restricted to "this month" — both must apply together, and the count threshold must be checked after grouping.

Key Issues Identified: 1) Filtering on month must happen before/alongside grouping (on the raw ApptDate), while the count threshold must be applied with HAVING after grouping. 2) A doctor can only be counted using appointments in the current month, not their whole history. 3) DoctorName needs a join back to Doctor since Appointment only stores DoctorID.

```
DOCTOR(DoctorID PK, DoctorName, Specialization)
PATIENT(PatientID PK, PatientName)
APPOINTMENT(ApptID PK, DoctorID FK->DOCTOR, PatientID FK->PATIENT, ApptDate)
```

Application of Concepts — Schema:

Sample data — Appointment (11 rows, August 2026 dates for Dr. Kumar to exceed the threshold)

ApptID	DoctorID	ApptDate
A1	D1	2026-08-01
A2	D1	2026-08-02
A3	D1	2026-08-03
A4	D1	2026-08-04
A5	D1	2026-08-05
A6	D1	2026-08-06
A7	D1	2026-08-07
A8	D1	2026-08-08
A9	D1	2026-08-09
A10	D1	2026-08-10
A11	D1	2026-08-11
A12	D2	2026-08-01
A13	D2	2026-08-02

```
SELECT D.DoctorName, COUNT(*) AS AppointmentCount
FROM Appointment A
JOIN Doctor D ON A.DoctorID = D.DoctorID
WHERE MONTH(A.ApptDate) = MONTH(CURRENT_DATE)
AND YEAR(A.ApptDate) = YEAR(CURRENT_DATE)
GROUP BY D.DoctorName
HAVING COUNT(*) > 10;
```

Solution:

Result — only Dr. Kumar clears the >10 threshold

DoctorName	AppointmentCount
------------	------------------

Dr. Kumar (D1)	11
----------------	----

Decision Making: The month/year filter is placed in WHERE (a per-row condition on raw ApptDate) while the >10 condition is placed in HAVING (a condition on the aggregated COUNT) — mixing these up would either be a syntax error or silently count appointments from all months, which is a common mistake this design deliberately avoids.

Clarity: The result is a short, ranked list a hospital administrator could act on directly — Dr. Kumar is over capacity this month while Dr. Reddy (D2, 2 appointments) is not.

Q3. E-commerce - Products Never Ordered

Scenario: An online store wants to identify dead stock — products that have never appeared in any customer order — to review pricing or discontinue them.

Understanding of Scenario: "Never ordered" is a negative condition — it asks for products with NO matching row in OrderItem, which is naturally expressed as a sub-query using NOT IN / NOT EXISTS rather than a positive join condition.

Key Issues Identified: 1) A product can appear zero, one, or many times across OrderItem rows; existence, not count, is what matters here. 2) NOT IN is unsafe if the sub-query can return NULL ProductIDs, so NOT EXISTS (or a NOT NULL-guarded NOT IN) is the safer method. 3) The comparison must be against the OrderItem table, not Orders, since Orders alone doesn't say which products were bought.

```
PRODUCT(ProductID PK, ProductName, Price)
ORDERS(OrderID PK, CustomerID, OrderDate)
ORDERITEM(OrderItemID PK, OrderID FK->ORDERS, ProductID FK->PRODUCT, Qty)
```

Application of Concepts — Schema:

Sample data — Product

ProductID	ProductName
P1	Wireless Mouse
P2	USB-C Hub
P3	Laptop Stand
P4	Webcam Cover (never ordered)

Sample data — OrderItem

OrderItemID	OrderID	ProductID
OI1	O1	P1
OI2	O1	P2
OI3	O2	P1

Solution:

```
SELECT ProductName
FROM Product
WHERE ProductID NOT IN (
    SELECT ProductID FROM OrderItem WHERE ProductID IS NOT NULL
);
```

-- Equivalent, safer against NULLs, and often better-performing:

```
SELECT P.ProductName
FROM Product P
WHERE NOT EXISTS (
    SELECT 1 FROM OrderItem OI WHERE OI.ProductID = P.ProductID
);
```


Result

ProductName
Laptop Stand
Webcam Cover (never ordered)

Decision Making: NOT EXISTS is preferred over plain NOT IN because NOT IN silently returns an empty result set the instant the sub-query produces even a single NULL — a subtle bug that NOT EXISTS never has, since it only tests row existence and is unaffected by NULLs in the compared column.

Clarity: Laptop Stand and Webcam Cover surface as dead stock (P3, P4 never appear in OrderItem), directly answering the business question of which products to review.

Q4. University -Relational Algebra for Students Enrolled in Both DBMS and OS

Scenario: A university wants to identify students who are taking two specific courses simultaneously, e.g. to check prerequisite overlaps or timetable clashes.

Understanding of Scenario: "Enrolled in both" is a set-intersection requirement over two independently-computed sets of SIDs — one set per course — which maps directly onto the relational algebra $\cap$ operator, not $\cup$ (which would mean "either course").

Key Issues Identified: 1) The two candidate sets (DBMS-takers, OS-takers) must be built the same way — projected onto SID only — so they are union/intersection-compatible. 2) Selecting on CourseName requires joining Enrollment with Course first, since Enrollment alone only stores CourseID.

Enrollment(SID, CourseID)
Course(CourseID, CourseName)

Application of Concepts — Schema:

Sample data — Course

CourseID	CourseName
C1	DBMS
C2	OS
C3	CN

Sample data — Enrollment

SID	CourseID
ST1	C1
ST1	C2
ST2	C1

ST2	C3
ST3	C2

$DBMS_Students \leftarrow \pi_{SID} (\sigma_{CourseName='DBMS'} (Enrollment \bowtie Course))$

$OS_Students \leftarrow \pi_{SID} (\sigma_{CourseName='OS'} (Enrollment \bowtie Course))$

$Result \leftarrow DBMS_Students \cap OS_Students$

Solution:

DBMS_Students

SID

ST1
ST2

OS_Students

SID
ST1
ST3

$$\text{Result} = \text{DBMS_Students} \cap \text{OS_Students}$$

SID
ST1

Decision Making: $\cap$ was chosen over a join-based approach because it operates on two already-independent projections of SID — this keeps the two conditions (CourseName='DBMS' and CourseName='OS') cleanly separated instead of trying to match a single Enrollment row against two course names at once, which is impossible since each Enrollment row records only one course.

Clarity: Only ST1 appears in both sets, so ST1 is the sole student taking both DBMS and OS — ST2 (DBMS+CN) and ST3 (OS only) are correctly excluded.

Q5. Payroll -View for Below-Average Earners + 10% Raise

Scenario: HR wants to give a targeted 10% raise to employees currently earning below their own department's average, as a corrective adjustment.

Understanding of Scenario: "Below department average" is department-scoped, not company-scoped, so the view's underlying comparison must be correlated per department, similar in spirit to the correlated sub-query pattern. The view and the UPDATE are two separate steps — the view identifies who qualifies, the UPDATE acts on them.

Key Issues Identified: 1) The comparison salary must be recomputed per department (correlated sub-query), not one global AVG(Salary). 2) The UPDATE must apply the 10% raise only to employees currently identified by the view — running it naively could raise everyone. 3) Because AVG shifts once salaries change, the view's WHERE clause needs to be evaluated once, up front, before the UPDATE, not recomputed mid-update.

Application of Concepts — Schema:

Employee(EID, EName, DeptID, Salary)

Sample data — Employee

EID	EName	DeptID	Salary
E01	Arun	D01	40000
E02	Priya	D01	60000
E03	Karthik	D02	50000

E04	Divya	D02	50000
-----	-------	-----	-------

```
CREATE VIEW BelowAvgSalary AS
SELECT E1.EID, E1.ENAME, E1.DeptID, E1.Salary
FROM Employee E1
WHERE E1.Salary < (
    SELECT AVG(E2.Salary) FROM Employee E2 WHERE E2.DeptID = E1.DeptID
);
```

Solution:

```
-- Give the qualifying employees a 10% raise
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EID IN (SELECT EID FROM BelowAvgSalary);
```

Contents of BelowAvgSalary before the UPDATE (D01 avg=50000, D02 avg=50000)

EID	EName	DeptID	Salary
E01	Arun	D01	40000

Employee table after the UPDATE

EID	EName	DeptID	Salary
E01	Arun	D01	44000
E02	Priya	D01	60000
E03	Karthik	D02	50000
E04	Divya	D02	50000

Decision Making: Only Arun ($\text{₹}40,000 < \text{D01's ₹}50,000$ average) qualifies — Karthik and Divya sit exactly at D02's average, not below it, so a strict $<$ (not $\leq$) correctly excludes them. Selecting EIDs from the view first, then updating by ID, avoids the ambiguity of re-evaluating a shifting AVG mid-statement.

Clarity: The view can be reused any time HR needs this list, and the single UPDATE statement makes the salary correction auditable and repeatable rather than a manual, error-prone process.

Q6. Banking-JOINs Across Customer, Account, Transaction, Branch

Scenario: A bank wants a single consolidated report combining customer identity, their account, recent transactions, and which branch holds the account.

Understanding of Scenario: Four entities are chained by foreign keys: Customer→Account (one customer can have many accounts), Account→Transaction (one account has many transactions), and Account→Branch (each account belongs to one branch). All must be joined together for a single readable report row.

Key Issues Identified: 1) The join order matters for readability but not correctness — all four tables share a chain of FKs so a straightforward sequence of INNER JOINs suffices. 2) An account could theoretically have zero transactions yet the report may still need to list it — that would call for a LEFT JOIN on Transaction specifically, which is noted as a design variant below.

```
BRANCH(BranchID PK, BranchName, City)
CUSTOMER(CustomerID PK, CustomerName)
ACCOUNT(AccountNo PK, CustomerID FK->CUSTOMER, BranchID FK->BRANCH, Balance)
TRANSACTION(TxnID PK, AccountNo FK->ACCOUNT, Type, Amount, TxnDate)
```

Application of Concepts — Schema:

Sample data — Branch

BranchID	BranchName	City
B1	Anna Nagar	Chennai

Sample data — Customer / Account

CustomerName	AccountNo	BranchID	Balance
Rahul	AC1001	B1	25000

Sample data — Transaction

TxnID	AccountNo	Type	Amount
-------	-----------	------	--------

TX1	AC1001	Deposit	5000
TX2	AC1001	Withdrawal	2000

```
SELECT C.CustomerName, A.AccountNo, A.Balance,
       T.Type, T.Amount, T.TxnDate,
       Br.BranchName, Br.City
FROM   Customer C
JOIN   Account A   ON C.CustomerID = A.CustomerID
JOIN   Transaction T ON A.AccountNo = T.AccountNo
JOIN   Branch Br   ON A.BranchID = Br.BranchID
WHERE  C.CustomerName = 'Rahul';
```

Solution:

Result

CustomerName	AccountNo	Balance	Type	Amount	BranchName
Rahul	AC1001	25000	Deposit	5000	Anna Nagar
Rahul	AC1001	25000	Withdrawal	2000	Anna Nagar

Decision Making: Plain INNER JOINs were used throughout because the scenario's chain (Customer→Account→Transaction/Branch) is expected to be fully populated for an active account; if the bank also needed to report brand-new accounts with zero transactions yet, JOIN Transaction would be changed to LEFT JOIN Transaction so those accounts aren't dropped from the report.

Clarity: Each output row reads as one transaction with its full context (who, which account, which branch, what happened) — exactly the flat report format a bank statement or audit view needs.

Q7. Retail - Daily Sales Summary by Category (GROUP BY + HAVING)

Scenario: A retail store wants a report of which product categories are performing well, defined as total sales exceeding ₹10,000.

Understanding of Scenario: This is a straightforward grouped-aggregate-with-threshold problem: total sales must be summed per category, then only categories clearing ₹10,000 should be reported.

Key Issues Identified: 1) SUM(Amount) is an aggregate, so the ₹10,000 threshold must go in HAVING, not WHERE. 2) "Daily" summary implies the report may also need to be scoped to a specific date if the store wants a single day's figures — shown as an optional WHERE addition.

Application of Concepts — Schema:

```
Sales(SaleID, Category, Amount, SaleDate)
```

Sample data — Sales (single day)

SaleID	Category	Amount
S1	Electronics	6000
S2	Electronics	5500
S3	Groceries	3000
S4	Groceries	2500

S5	Stationery	1200
----	------------	------

```
SELECT Category, SUM(Amount) AS TotalSales
FROM Sales
WHERE SaleDate = CURRENT_DATE      -- optional: restrict to "today"
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

Solution:

Grouped totals before HAVING

Category	TotalSales
Electronics	11500
Groceries	5500
Stationery	1200

Final result after HAVING > 10000

Category	TotalSales
Electronics	11500

Decision Making: The optional WHERE SaleDate = CURRENT_DATE line scopes the summary to "daily" as the scenario specifies, while keeping it separate from HAVING preserves the correct order of operations: filter rows first (WHERE), aggregate, then filter groups (HAVING).

Clarity: Electronics alone crosses ₹10,000 (₹11,500) while Groceries and Stationery fall short — the report tells the store manager exactly which category to feature.

Q8. Library -TRC for Members Who Borrowed From All Genres

Scenario: A library wants to recognise members who have sampled books from every genre in the catalogue.

Understanding of Scenario: "All genres" is a universal-quantification requirement, the tuple-calculus counterpart of relational algebra's division — it must be phrased with a $\forall$ (for all genres, some matching borrow record exists), not a simple $\exists$ (which would mean "at least one genre").

Key Issues Identified: 1) The predicate needs two nested quantifiers: $\forall$ over genres, and $\exists$ over Book/Borrow to witness that the member borrowed a book of that genre. 2) Genre is a property of Book, not of Borrow directly, so the witness must join Borrow to Book on BookID.

```
Member(MemberID, MemberName)
Book(BookID, Title, Genre)
Borrow(MemberID, BookID)
```

Application of Concepts — Schema:

Sample data — Book (3 genres offered)

BookID	Genre
B1	Fiction
B2	Science
B3	History

Sample data — Borrow

MemberID	BookID
M1	B1
M1	B2

M1	B3
M2	B1
M2	B2

```

{ t | t ∈ Member ∧
  ∀ g ( g ∈ Book ⇒
    ∃ br ( br ∈ Borrow ∧ br.MemberID = t.MemberID ∧
      ∃ b ( b ∈ Book ∧ b.BookID = br.BookID ∧ b.Genre = g.Genre ) ) ) }

```

Solution:

Working through the quantifiers by hand:

Genres each member's borrow history covers, vs. the full genre set {Fiction, Science, History}

MemberID	Genres borrowed	Covers all 3?
M1	Fiction, Science, History	Yes
M2	Fiction, Science	No — missing History

Final Result

MemberID
M1

Decision Making: The $\forall \dots \Rightarrow \exists \dots$ nesting was chosen specifically because "all genres" cannot be captured by a single $\exists$ — a member with just one borrow record would trivially satisfy an unqualified existential check, which is exactly the wrong answer; the $\forall$ forces every genre in the catalogue to be individually witnessed.

Clarity: M1 borrowed at least one book in each of the three genres and is returned; M2, missing History entirely, is correctly excluded despite having borrowed from two of the three genres.

Q9. Inventory - CHECK and FOREIGN KEY Constraint Enforcement

Scenario: An inventory system must prevent physically impossible stock levels (negative quantity) and must not let a product reference a category that doesn't exist.

Understanding of Scenario: Two distinct integrity concerns are being asked for: a domain rule local to one table (CHECK on Quantity) and a cross-table referential rule (FOREIGN KEY on CategoryID) — both need to be defined and then demonstrated actually rejecting bad data, not just declared.

Key Issues Identified: 1) CHECK constraints are evaluated per-row at INSERT/UPDATE time and reject the statement outright if violated. 2) FOREIGN KEY constraints require the referenced CategoryID to pre-exist in Category; violating it should also cause the whole INSERT to fail. 3) "Demonstrate enforcement" means showing both a failing statement and the constraint error it triggers, alongside a valid one that succeeds.

```

CATEGORY(CategoryID PK, CategoryName)
PRODUCT(ProductID PK, ProductName, Quantity CHECK (Quantity >= 0),
        CategoryID FK -> CATEGORY(CategoryID))
CREATE TABLE Category (
    CategoryID VARCHAR(10) PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);

CREATE TABLE Product (
    ProductID VARCHAR(10) PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity >= 0),
    CategoryID VARCHAR(10),
    CONSTRAINT fk_product_category
        FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

```

Application of Concepts — Schema:

Sample data — Category

CategoryID	CategoryName
CAT1	Stationery

```
-- (a) Valid insert — succeeds
INSERT INTO Product VALUES ('P1', 'Notebook', 50, 'CAT1');

-- (b) CHECK violation — negative quantity, REJECTED
INSERT INTO Product VALUES ('P2', 'Broken Stapler', -5, 'CAT1');
-- Error: new row violates check constraint "product_quantity_check"

-- (c) FOREIGN KEY violation — CategoryID does not exist, REJECTED
INSERT INTO Product VALUES ('P3', 'Mystery Item', 10, 'CAT9');
-- Error: insert or update on table "product" violates foreign key constraint
--      "fk_product_category"
```

Demonstrating enforcement:

Resulting Product table (only the valid row (a) persisted)

ProductID	ProductName	Quantity	CategoryID
P1	Notebook	50	CAT1

Decision Making: CHECK (Quantity >= 0) is enforced at the column level so it applies to every future INSERT/UPDATE automatically, rather than relying on application code to validate — this centralises the rule in the database, which is the more robust design when multiple applications might write to the same table.

Clarity: Statements (b) and (c) both fail before any row is written, so the Product table never reaches an inconsistent state — exactly what "enforcement" (as opposed to just declaring the constraint) means.

Q10. HR System - Schema + Materialized Views for Attendance and Performance

Scenario: An HR department wants fast, pre-computed monthly summaries of attendance and performance for reporting, without recomputing aggregates from raw logs on every request.

Understanding of Scenario: Two different summary reports are needed (attendance, performance), each aggregated by employee and month. Because these are read often but the underlying data changes relatively slowly (once a day/month), materialized views — which physically store the computed result rather than recomputing it on every query — are the right tool, as the question specifically asks for.

Key Issues Identified: 1) Attendance and Performance are separate source tables and should produce two separate materialized views, not one merged view, since they aggregate on different grains of data. 2) A materialized view's data goes stale as new attendance/performance rows are added, so a REFRESH step must be part of the design, not an afterthought.

```
EMPLOYEE(EID PK, EName, DeptID)
ATTENDANCE(AttID PK, EID FK->EMPLOYEE, AttDate, Status) -- Status: Present/Absent/Leave
PERFORMANCE(PerfID PK, EID FK->EMPLOYEE, ReviewMonth, Rating)
```

Application of Concepts — Schema:

Sample data — Attendance (August 2026)

EID	AttDate	Status
E1	2026-08-01	Present
E1	2026-08-02	Present
E1	2026-08-03	Absent
E2	2026-08-01	Present
E2	2026-08-02	Leave

```

CREATE MATERIALIZED VIEW MonthlyAttendanceSummary AS
SELECT EID,
       DATE_TRUNC('month', AttDate) AS Month,
       COUNT(*) FILTER (WHERE Status = 'Present') AS DaysPresent,
       COUNT(*) FILTER (WHERE Status = 'Absent') AS DaysAbsent,
       COUNT(*) FILTER (WHERE Status = 'Leave') AS DaysOnLeave
FROM Attendance
GROUP BY EID, DATE_TRUNC('month', AttDate);

CREATE MATERIALIZED VIEW MonthlyPerformanceSummary AS
SELECT EID, ReviewMonth, AVG(Rating) AS AvgRating
FROM Performance
GROUP BY EID, ReviewMonth;

-- Refresh periodically (e.g. nightly job) as new rows arrive:
REFRESH MATERIALIZED VIEW MonthlyAttendanceSummary;
REFRESH MATERIALIZED VIEW MonthlyPerformanceSummary;

```

Solution:

Contents of MonthlyAttendanceSummary for August 2026

EID	Month	DaysPresent	DaysAbsent	DaysOnLeave
E1	2026-08	2	1	0
E2	2026-08	1	0	1

Decision Making: A materialized view was chosen over a normal (virtual) view specifically because the question emphasises a reporting workload — HR dashboards querying these summaries repeatedly would otherwise re-run the full GROUP BY over raw Attendance/Performance rows every single time; materializing trades a small amount of staleness (fixed by REFRESH) for much faster read performance, which is the correct trade-off for a monthly summary that doesn't need to be real-time.

Clarity: The two views cleanly separate concerns — one dashboard widget can pull attendance, another performance — each already pre-aggregated to the EID + month grain the HR reports need.